

Godot Terrain3D Texture System (32-Texture) and Three.js Implementation

Overview of Terrain3D Texture System

Terrain3D (TokisanGames) is a high-performance terrain plugin for Godot that supports **up to 32 texture sets** per terrain ¹ ². Each *texture set* contains an albedo (RGB) and normal (RGB) image plus auxiliary data: the albedo image's alpha channel holds a height map, and the normal image's alpha holds a roughness map ³. All albedo images must be the same size, and all normal images likewise; these two groups are combined into two separate `Texture2DArray` objects on the GPU ⁴. In practice, a single terrain thus uses one 2D texture array for color+height and another for normal+roughness.

Terrain painting is done per-vertex (one paint sample per vertex). Each terrain vertex stores a **base texture ID, an overlay texture ID, and a blend weight**. These values are written into a *control map* (sometimes called an index map) that is sampled by the shader. In effect, each vertex can reference two of the 32 textures and blend between them. Between vertices, the shader bilinearly interpolates these values to smoothly mix the textures. This approach differs from typical "splatmap" terrain (which stores per-pixel blending weights in multiple RGBA maps) and enables a much more compact encoding of up to 32 textures.

Internal Shader and Material Structure

Terrain3D provides a custom `Terrain3DMaterial` (written in Godot's shader language) and a C++ GDExtension backend. The material exposes uniforms for terrain data (regions, height maps, control maps, color maps) and the texture arrays. Key uniforms include `_height_maps`, `_control_maps`, `_texture_array_albedo`, and `_texture_array_normal` ⁵. Internally, a clipmap mesh is generated on the CPU; in the **vertex shader**, each vertex looks up the control map to determine its base/overlay texture IDs, blend factor, hole flags, etc., and also samples the heightmap for the vertex's elevation (skipping GPU vertex transform) ⁵. The vertex shader thus adjusts the mesh to match the terrain height.

In the **fragment shader**, the terrain is rendered per-pixel. For each pixel, the shader samples the control map at the four surrounding vertices (a 2x2 grid). Each sampled point provides two texture indices (0-31) and an 8-bit blend value. The shader then fetches the corresponding textures from the `Texture2DArray`s: e.g. `texture(albedo_array, vec3(UV, base_id))` and `texture(albedo_array, vec3(UV, overlay_id))`, and similarly from the normal/roughness array. These samples are blended together using the blend weight and bilinear interpolation. Additional factors like each texture's height map are used to refine the mix (for example, to let higher "height" textures overlap others) ⁶. The result is then combined with PBR lighting (using the normals, roughness, etc.) to produce the final surface.

Use of Texture Arrays and Index Maps

Texture Arrays: Because all albedo textures are the same size, Terrain3D places them into a single GPU texture array (`sampler2DArray`) called `_texture_array_albedo` . Likewise, all normal textures (same dimensions) go into `_texture_array_normal` . In GLSL these are accessed via `texture(texture_array, vec3(uv, layer))` , where `layer` is the texture ID. This lets the shader index any of the 32 textures with one sampler. (Godot's engine generates these arrays at import or runtime from the individual image files.)

Index/Control Maps: Terrain3D uses a packed control map (one texel per terrain vertex). Each texel is a 32-bit integer encoding several fields: a 5-bit base texture ID (0–31), a 5-bit overlay texture ID, an 8-bit blend factor (0–255), plus extra bits for UV rotation, scale, hole flags, etc. ⁷ ⁸ . For example, bits 32–28 store the base ID, bits 27–23 the overlay ID, and bits 22–15 the blend (see table in control map docs). Godot stores this as a float texture (`FORMAT_RF`) and then uses integer bit ops in the shader to decode it.

At render time, the shader samples four neighboring control texels (the grid of surrounding vertices). Each supplies a base ID, overlay ID, and blend. The pixel's final blend is computed by bilinearly interpolating these values based on the pixel's position within the vertex quad. In effect, even though each vertex only references two textures, interpolation allows up to **4 textures** to influence a pixel. The blend factor and height maps further modulate how the textures mix. This scheme, borrowed from the Witcher 3 and other advanced terrain techniques, uses only two texture IDs per vertex but yields smooth multi-texture blends ⁹ ¹⁰ . Height-based blending is supported: each texture's alpha channel (height map) is used in the blend to give a natural overlap (e.g. snow sits on top of grass) ⁶ .

Efficiency vs Traditional Splatmaps

Compared to classic splatmap systems, Terrain3D's index-map method is far more efficient in memory and often in lookups. Traditional splatting stores, for each pixel, one byte of weight per texture per channel. For 16 textures you need 4 RGBA maps (16 bytes per pixel), and for 32 textures you need 8 maps (32 bytes/pixel). On a 4096×4096 terrain (≈16M pixels) that is **256MB for 16 textures or 512MB for 32 textures** ¹¹ . By contrast, Terrain3D's control map uses only 18 bits per pixel (5+5+8) for 32 textures, requiring **~2.25 bytes per pixel**. A 4K terrain with 32 textures then uses only about **36MB** of control data – a ~93% VRAM reduction ¹¹ .

In terms of texture fetches: splat shaders typically do 4–8 lookups to find the top N weights and then 4 texture samples, totaling ~12–16 texelFetch calls per pixel ¹² . Terrain3D, by sampling its 4 control points and then 2–4 textures per point, does roughly 5–20 lookups depending on distance (at far LOD it may skip some bilinear blends) ¹² . Thus Terrain3D often cuts memory bandwidth and draw calls. The trade-off is that a splatmap can blend many layers per pixel (subject to GPU limitations), while Terrain3D blends two per vertex (up to four via interpolation). In practice this still yields very smooth results with proper height maps ¹⁰ .

Method	Memory per pixel (4K terrain)	Texel Lookups	Blended textures
Splatmaps	16 bytes (16 tex) / 32 bytes (32 tex) ⇒ ~256MB / 512MB ¹¹	~12–16 (IDs + texture fetches) ¹³	Up to 4 per pixel (weights stored in RGBA maps)
Terrain3D	~2.25 bytes (18 bits, 32 tex) ⇒ ~36MB ¹¹	~5–20 (4 control samples + 2–4 texture samples) ¹³	2 per vertex (4 via interpolation, with height-based overlap)

How to Replicate This in Three.js

To implement a similar 32-texture terrain in Three.js, you must use WebGL2 features (texture arrays) or carefully work around them:

- **Prepare Texture Arrays:** In a WebGL2 context, load your 32 albedo+height images (all the same size) into a single `THREE.DataTexture2DArray` (or equivalent). For example:

```
const width = 1024, height = 1024, depth = 32;
// Suppose `pixelData` is a Uint8Array of size width*height*4*depth
// containing all images in sequence.
const textureArray = new THREE.DataTexture2DArray(pixelData, width,
height, depth);
textureArray.format = THREE.RGBAFormat;
textureArray.needsUpdate = true;
```

Do the same for your normal+roughness textures in a second array. This gives you `uniform sampler2DArray textureAlbedoArray;` and `textureNormalArray` in the shader.

- **Build a Control Texture:** Create a single 2D texture (RGBA8) where each texel encodes one vertex's data: use R = base texture ID (0–255), G = overlay ID (0–255), A = blend (0–255). (If needed, you could pack two 5-bit IDs and an 8-bit blend into RGBA channels; simpler is to just use R,G,A and reserve B.) Upload this as a normal `THREE.Texture`.
- **Shader Logic:** In your `ShaderMaterial`, sample the control map and texture arrays. For example, in the fragment shader:

```
uniform sampler2DArray textureAlbedoArray;
uniform sampler2D controlMap;
varying vec2 vUv;

void main() {
    vec4 ctrl = texture(controlMap, vUv);
    int baseId = int(floor(ctrl.r * 255.0)); // base texture index
```

```

int overlayId = int(floor(ctrl.g * 255.0)); // overlay texture index
float blend = ctrl.a;                      // blend factor [0,1]
vec4 baseColor = texture(textureAlbedoArray, vec3(vUv, float(baseId)));
vec4 overlayColor = texture(textureAlbedoArray, vec3(vUv,
float(overlayId)));
// Optionally use height maps: e.g. float baseHeight = baseColor.a;
vec4 color = mix(baseColor, overlayColor, blend);
gl_FragColor = color;
}

```

In Three.js, set these up in JavaScript:

```

const material = new THREE.ShaderMaterial({
  uniforms: {
    textureAlbedoArray: { value: textureArray },
    controlMap: { value: controlTexture },
    // ... add textureNormalArray, etc.
  },
  vertexShader: /* simple pass-through computing vUv from vertex */,
  fragmentShader: /* as above */,
});

```

Ensure your UV mapping matches the control map (e.g. one UV unit per terrain tile). If you want height-based blending, you can unpack each texel's alpha (height) and use it to bias `blend` (for example, if overlay height > base height, favor overlay color).

- **Normal and Roughness:** Similarly sample a second `sampler2DArray` for normals/roughness. For instance, after computing `baseColor` and `overlayColor`, also do:

```

vec3 baseNormal = texture(textureNormalArray, vec3(vUv,
float(baseId))).rgb;
vec3 overlayNormal = texture(textureNormalArray, vec3(vUv,
float(overlayId))).rgb;
vec3 normal = normalize(mix(baseNormal, overlayNormal, blend));

```

Then use `normal` in your lighting calculation.

- **Texture Units:** WebGL2 allows many texture units, but Three.js may still have limits. Using texture arrays avoids consuming 32 sampler slots; each array is one sampler.

WebGL1 vs WebGL2 Considerations

WebGL2 is **required** to use true 2D texture arrays (`sampler2DArray`). In WebGL1, no native texture arrays exist. Workarounds include:

- **Texture Atlas:** Pack multiple textures into one large atlas (with careful UV offsetting and padding). This complicates filtering and requires more shader logic.
- **Uniform Texture Lists:** Use multiple `uniform sampler2D tex[32];` arrays (some platforms allow arrays of samplers), but dynamic indexing in GLSL ES2 is limited (indices often must be constant or loop-unrolled). And 32 separate textures may exceed the maximum texture units.
- **Fallback to Fewer Textures:** Limit yourself to <16 textures (the typical max texture units) in WebGL1.
- **Extensions:** WebGL1 on some devices supports `EXT_texture3D` (enabling `sampler3D` for 3D textures) or the older `WEBGL_texture_array` extension, but these are not universally available. Three.js by default creates a WebGL1 context if WebGL2 is not requested.

In practice, for a full 32-texture array approach, a WebGL2 context is strongly recommended. Three.js can be set to use WebGL2 by creating the renderer with `{ context: canvas.getContext('webgl2') }`. Without WebGL2, one must resort to atlases or multiple shaders.

Sample Three.js ShaderMaterial Snippet

Below is a minimal illustrative snippet of how you might set up the ShaderMaterial and GLSL code. Note this is a conceptual example:

```
// Assume textureArray and controlTexture are already created as above.
const terrainMaterial = new THREE.ShaderMaterial({
  uniforms: {
    textureAlbedoArray: { value: textureArray },
    textureNormalArray: { value: textureNormalArray },
    controlMap: { value: controlTexture },
    // ... other uniforms like repeat scales, lighting, etc.
  },
  vertexShader: `
    varying vec2 vUv;
    void main() {
      vUv = uv; // or compute from position
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position,1.0);
    }
  `,
  fragmentShader: `
    uniform sampler2DArray textureAlbedoArray;
    uniform sampler2DArray textureNormalArray;
    uniform sampler2D controlMap;
    varying vec2 vUv;
    void main() {
      // Sample control map (RGBA: R=baseID/255, G=overlayID/255, A=blend)
      vec4 ctrl = texture(controlMap, vUv);
      int baseId = int(floor(ctrl.r * 255.0));
      int overlayId = int(floor(ctrl.g * 255.0));
      float blend = ctrl.a;
```

```

        // Fetch albedo and height
        vec4 baseTex    = texture(textureAlbedoArray, vec3(vUv, float(baseId)));
        vec4 overlayTex = texture(textureAlbedoArray, vec3(vUv,
float(overlayId)));
        // Fetch normals (rgb) and roughness (alpha) similarly
        vec3 baseNormal    = texture(textureNormalArray, vec3(vUv,
float(baseId))).rgb;
        vec3 overlayNormal = texture(textureNormalArray, vec3(vUv,
float(overlayId))).rgb;

        // Height-based adjust (if desired):
        // float baseH = baseTex.a, overlayH = overlayTex.a;
        // blend = (overlayH > baseH) ? 1.0 : blend;

        // Blend colors and normals
        vec4 color  = mix(baseTex, overlayTex, blend);
        vec3 normal = normalize(mix(baseNormal, overlayNormal, blend));

        // Simple diffuse shading example:
        float light = dot(normal, vec3(0,0,1))*0.5 + 0.5;
        gl_FragColor = vec4(color.rgb * light, 1.0);
    }
},
});

```

In the code above, `textureAlbedoArray` is a `THREE.DataTexture2DArray`, and `controlMap` is a regular `THREE.Texture` with encoded data. The fragment shader decodes texture indices and blends the two textures. You would expand this with your actual lighting, normal mapping, and any macros or color variation (e.g. a color map multiplied onto albedo).

Note on Three.js versions: As of writing, Three.js supports `DataTexture2DArray`, but building it from image files typically requires combining them into raw pixel data (or using intermediate canvas operations). Check the Three.js docs for `DataTexture2DArray` usage. The key idea is that the GLSL code uses `sampler2DArray` to index textures by layer, mirroring Godot's approach.

Sources

This breakdown is based on the official Terrain3D documentation and discussions [1](#) [2](#) [9](#) [13](#) [11](#) [7](#) [6](#), which detail the texture packing, shader logic, and memory trade-offs of Godot's system. These techniques (texture arrays, index maps, height-based blending) can be re-implemented in WebGL2 via Three.js as illustrated above.

[1](#) [3](#) [4](#) Preparing Textures — Terrain3D 1.0.1 documentation
https://terrain3d.readthedocs.io/en/stable/docs/texture_prep.html

2 Texturing the Terrain — Terrain3D 1.0.1 documentation

https://terrain3d.readthedocs.io/en/stable/docs/texture_painting.html

5 terrain3d.readthedocs.io

https://terrain3d.readthedocs.io/en/stable/_sources/docs/shader_design.md.txt

6 Technical Tips — Terrain3D 1.0.1 documentation

https://terrain3d.readthedocs.io/en/stable/docs/tips_technical.html

7 8 Control Map Format — Terrain3D 1.0.1 documentation

https://terrain3d.readthedocs.io/en/stable/docs/controlmap_format.html

9 10 11 12 13 Shader Design — Terrain3D 1.0.1 documentation

https://terrain3d.readthedocs.io/en/stable/docs/shader_design.html